Bibliotequinha da Rhay 2.0

Relatório técnico de implementação e validação

Autora do projeto: Rhayanna. Projeto individual, com assistência técnica de Codex na implementação e validação descritas. Data verificada desta execução: 02/11/2026. Não há prazo de entrega confirmado. O resumo do enunciado indica grupos de 3 a 4 pessoas; confirmar autorização de entrega individual com o professor. Não foram inventados colegas ou contribuições no Git.

Tema e objetivo

Sistema web de biblioteca para organizar exemplares físicos, categorias, usuários e empréstimos, com acesso por perfil. Cada livro cadastrado representa um exemplar identificado por código único. A versão anterior já fornecia autenticação Breeze e CRUD de acervo; a versão 2.0 acrescenta empréstimos, API REST e programação PostgreSQL.

Metodologia

O repositório real foi inspecionado antes das alterações: dependências, rotas, modelos, migrations, seeders, Controllers, Form Requests, Policies, Blade, testes e configuração. O trabalho foi isolado na branch feat/bibliotequinha-segundo-bimestre. Arquivos não rastreados e entidades acadêmicas anteriores foram preservados. Não houve push ou publicação.

Foram implementadas migrations incrementais, Services, autorização e API; testes reais orientaram correções. O banco atual recebeu backup privado antes da aplicação das duas migrations novas, sem exclusão de registros. Testes de reconstrução e restauração usaram instância descartável separada.

Arquitetura e segurança

Laravel 12.69.2, PHP 8.2.12, PostgreSQL 18, Blade, Breeze, Eloquent, Vite e PHPUnit. Controllers coordenam requisições; Form Requests validam; Policies autorizam; Services centralizam regras e chamadas SQL; Resources definem o JSON público. Categorias possuem livros e usuários/livros possuem empréstimos.

Papéis existentes preservados: admin, bibliotecario e consulta (leitor). O leitor só consulta seus próprios empréstimos. Cadastro público não permite escolher role. Tokens Bearer aleatórios são armazenados como hash e expiram em sete dias. Histórico impede exclusão de exemplares/contas referenciados. Consultas SQL usam parâmetros.

Programação SQL aplicada

Dois índices de consulta foram acrescentados: ordenação por título/id e empréstimos ativos por usuário/prazo. Um índice único parcial adicional garante apenas um ativo por exemplar. Funções de negócio retornam quantidade de ativos e disponibilidade. Procedures registram empréstimo e devolução com CALL. Triggers validam datas, disponibilidade e limite, auditam operações e sincronizam o exemplar; uma terceira protege a situação do livro.

LoanService chama SELECT e CALL dentro de transações. Locks serializam validações de usuário/exemplar. Functions auxiliares de trigger não são contadas como substitutas das duas functions de negócio. Não se promete ganho em tabelas pequenas: o EXPLAIN de 16 livros escolheu leitura sequencial, enquanto a consulta de ativos usou o índice parcial.

Resultados verificáveis

A suite completa em PostgreSQL isolado aprovou 45 testes e 243 assertions. Inclui rotas web, CRUD API, autenticação, autorização, empréstimo duplicado, limite, devolução repetida, chamadas SQL e auditoria automática. A suite SQLite aprovou 41 testes e pulou explicitamente os quatro testes PostgreSQL; não substitui a execução no banco correto. O build de produção Vite concluiu com 59 módulos e as 43 rotas da aplicação foram listadas.

Backup customizado com pg_dump foi restaurado com pg_restore em banco local novo e vazio. Origem e destino continham 3 usuários, 16 livros, 4 categorias, 1 empréstimo e 1 auditoria. Dados foram conferidos por assinatura idêntica. A devolução no destino restaurado funcionou e produziu auditoria UPDATE. Evidências reais estão em docs/evidencias/postgres-validacao.txt; dumps privados permanecem ignorados pelo Git.

Decisões e limites

Autor permanece campo textual para evitar entidade redundante. Empréstimos não admitem exclusão nem alteração de tomador/exemplar: a atualização é a devolução. Registros antigos marcados emprestado sem tomador são preservados para reconciliação manual. Seeders não sobrescrevem livros existentes nem redefinem senhas de contas existentes.

Neon foi conectado com SSL e ativado após backup de origem/destino: 7 usuários, 16 livros e 4 categorias transferidos e conferidos por assinatura. Duas functions, duas procedures, três triggers e três índices foram confirmados; retirada/devolução via Laravel passou em transação revertida, sem registros de teste persistidos. A configuração local foi preservada em .env.local, sem apagar o PostgreSQL local. O conector recebeu compatibilidade com libpq 11.4 do XAMPP; a suite atualizada passou com 46 testes e 248 assertions em banco local isolado. Evidências adicionais em docs/NEON.md. Não houve teste visual em navegador ou teste de carga simultânea. O enunciado original em PDF não foi localizado no workspace; a implementação seguiu o documento fornecido e o código existente.

Materiais de entrega

README, scripts SQL, diagnóstico, arquitetura, API, testes, backup/restauração, matriz de requisitos, roteiro de apresentação de 10 minutos e 32 perguntas de arguição. Este relatório em Markdown é a fonte do PDF gerado por scripts/generate-report.py.